Zephyr Project

IT-Schulungen.com
Technology & Transformation

C++-Übung mit native_sim

C-Schnittstelle kapseln und Auswirkungen im Build sichtbar machen

5.12

Übungsziel und Ausgangsstand

KOMPONENTE

C-API kapseln

Eine schmale C++-Klasse übernimmt Zugriff und Lebensdauer.

LAUFZEIT

Besitz klären

Initialisierung und Fehlerpfade bleiben explizit.

ANALYSE

Build vergleichen

C-Build

C++-Build

Größe und Symbole

Projektstand und Referenzlösung sind vorbereitet. Die Folien zeigen die technischen Entscheidungen.

Ablauf der Übung

Fünf Schritte vom C-API zum C++-Build

01

Schnittstelle

Signaturen lesen

02

Besitz

Lebensdauer festlegen

03

Kapseln

Wrapper schreiben

04

Integrieren

mit native_sim bauen

05

Vergleichen

Symbole und Größe

Der Code bleibt klein. Im Mittelpunkt stehen Sprachgrenze, Lebensdauer und Buildwirkung.

Bereitgestellte Projektstruktur

QUELLDATEIEN

```
src/sensor_api.c, src/sensor.hpp, src/main.cpp
```

C-Schnittstelle	sensor_api.h und sensor_api.c bilden die stabile Grenze
Wrapper	sensor.hpp kapselt Ressource und Fehlercode
Anwendung	main.cpp verwendet nur die C++-Schnittstelle
Build	CMakeLists.txt kompiliert C und C++ gemeinsam

Die Übung verändert die bereitgestellte C-API nicht.

Besitz und Lebensdauer

Besitz	Der Typ zeigt, ob er die Ressource besitzt oder nur referenziert
Erzeugen	Konstruktor oder Factory stellt einen gültigen Zustand her
Nutzen	Methoden bündeln Zephyr-Aufruf und Fehlerübersetzung
Kopieren	Kopieren verbieten oder Besitzübertragung ausdrücklich definieren
Freigeben	Lokale Objekte räumen Ressourcen am Scope-Ende auf

Besitz oder Referenz müssen in der Schnittstelle eindeutig erkennbar sein.

Konstruktor oder Factory

01	`Konstruktor`	Geeignet, wenn die Erzeugung nicht fehlschlagen kann
02	`Factory`	Gibt Status und gültiges Objekt getrennt zurück
03	`Verwendung`	main() entscheidet sichtbar über den Fehlerpfad

Für fehlbare Zephyr-Aufrufe bleibt die Initialisierung über Factory oder init()-Methode transparent.

Fehler typsicher behandeln

ERGEBNISTYP

```
enum class ReadStatus {
  ok, busy, io_error
};
```

01 **C-Fehlercode**

Der Wrapper übersetzt negative Rückgabewerte an einer Stelle.

02 **Ergebnisobjekt**

Status und Nutzwert bleiben gemeinsam prüfbar.

03 **Sprachgrenze**

Keine Exception überschreitet die C-API-Grenze.

Der Aufrufer behandelt erwartete Fehler explizit und ohne versteckten Kontrollfluss.

Compilezeit statt versteckter Kosten

STATISCHE KONFIGURATION

```
enum class Channel : uint8_t {
  accel, gyro
};
constexpr size_t sample_count = 32;
```

01 Starke Typen

enum class verhindert die implizite Vermischung fachlicher Werte.

02 Heap vermeiden

Feste Puffer und automatische Objekte machen den Speicherbedarf sichtbar.

constexpr und starke Typen verbessern die Schnittstelle ohne zusätzliche Laufzeitkosten.

Integration unter native_sim

KONFIGURATION

C++ aktiv

CONFIG_CPP=y

BUILD

Neu bauen

west build -p always -b native_sim

AUSFÜHRUNG

Verhalten prüfen

./build/zephyr/zephyr.exe

native_sim macht Sprachgrenze und Programmverhalten reproduzierbar auf dem Host sichtbar.

Symbole und Binärgröße vergleichen

WERKZEUGE

```
nm -C build/zephyr/zephyr.exe
size build/zephyr/zephyr.exe
```

01 Symbole

nm -C macht C++-Namen lesbar und zeigt den Wrapper im Image.

02 Größe

text, data und bss zwischen C- und C++-Variante vergleichen.

Die Messung zeigt den konkreten Effekt des Wrappers. Für ein MCU-Ziel ist eine eigene Messung erforderlich.